

06c — SSL/TLS

Key Concepts

- **TLS (Transport Layer Security)** — encryption protocol used by HTTPS
 - **SSL** — deprecated predecessor to TLS; names used interchangeably
 - **Asymmetric encryption** — two keys (public + private); slow; used for key exchange only
 - **Symmetric encryption** — one shared key; fast; used for actual data encryption
 - **Session key** — temporary shared key negotiated during handshake; used for symmetric encryption
 - **Certificate Authority (CA)** — trusted third party that signs certificates (DigiCert, Let's Encrypt)
 - **Self-signed certificate** — generated without a CA; browsers don't trust it automatically
-

TLS Handshake — Step by Step

1. Client → Server

"Hello, I support TLS 1.3, here are my cipher options"

2. Server → Client

Certificate (contains public key)

3. Client

Verifies certificate against trusted CA

4. Client → Server

Uses server's public key to negotiate a shared session key

5. Both sides

Derive the same symmetric session key

6. All further data

Encrypted with session key (AES – fast)

Key insight: Asymmetric encryption used once for key exchange only. Symmetric handles all actual data.

Asymmetric vs Symmetric

	Asymmetric (RSA)	Symmetric (AES)
Keys	Public + private pair	One shared key
Speed	Slow	Fast
Used for	Key exchange (handshake)	Encrypting data

TLS Versions

Version	Status
SSL 2.0 / 3.0	Deprecated — broken
TLS 1.0 / 1.1	Deprecated — disabled by browsers since 2020
TLS 1.2	Still widely used
TLS 1.3	Current standard — faster, stronger

CA-Signed vs Self-Signed Certificate

	CA-Signed	Self-Signed
Issued by	Trusted Certificate Authority	You
Browser trust	Automatic — no warnings	Warning shown
Use case	Production (public-facing)	Local dev, internal services

Certificate Chain of Trust

```
Root CA (DigiCert, Let's Encrypt)
├─ Intermediate CA
│   └─ Your server's certificate
```

Browsers ship with built-in list of trusted Root CAs. Certificate must trace back to one of them.

Industry Examples

- **Cloudflare** — terminates TLS at edge; origin server never handles raw TLS
- **Let's Encrypt** — free CA; made HTTPS accessible to all
- **Banks** — enforce TLS 1.2 minimum for PCI-DSS compliance

Interview Questions

Q: How does TLS work?

Browser connects, server sends certificate with public key, browser verifies with CA. Both sides use the public key to negotiate a shared session key (asymmetric). All actual data is then encrypted with that session key using fast symmetric encryption (AES).

Q: Why does TLS use both asymmetric and symmetric encryption?

Asymmetric is secure for key exchange but too slow for bulk data. Symmetric is fast but requires a shared key — you need asymmetric to safely establish it. TLS uses both: asymmetric once to exchange the key, symmetric for everything after.

Q: What is the difference between a CA-signed and self-signed certificate?

CA-signed traces back to a trusted root CA — browsers trust it automatically. Self-signed has no chain of trust — browsers show security warnings. Self-signed is fine for local dev or internal service communication but never for public-facing production.

Q: What TLS version should you use?

TLS 1.3 is the current standard — faster handshake and stronger security. Minimum TLS 1.2 for compliance. TLS 1.0/1.1 and all SSL versions are deprecated and should be disabled.

